

Two Implementations of Power-Spectrum Constraints

Where the constraint enters

- In `compsep_planck.py`, `ST_COMPUTE_PS=1` sets

`compute_PS=True`

when building the `ST_Operator`.

- The backend then appends a separate tensor `PS` to the flattened statistics:

`mean, var,  $S_1, \dots, S_4$ , PS.`

- The target and running maps are compared by the same squared Euclidean loss used for the other statistics.

Planck backend: channel pairs

In the 5-channel phase,

$$X = (d_Q, n_Q, d_U, n_U, d_I).$$

The power-spectrum operator uses the same upper-triangular pair mask as the scattering statistics:

$$\begin{pmatrix} 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

- Auto-spectra are computed for every channel.
- Cross-spectra are computed for

$$(d_Q, d_U), \quad (d_Q, d_I), \quad (d_U, d_I).$$

- Signal-noise and noise-auxiliary cross-spectra are not included.

Planck backend: what is computed

For each batch element, selected channel pair, and radial bin,

$$P_{c_1 c_2}(b) = \frac{\sum_k M_b(k) \hat{X}_{c_1}(k) \overline{\hat{X}_{c_2}(k)}}{\sum_k M_b(k)}.$$

- $M_b(k)$ is a smooth radial Fourier-bin mask.
- Output shape is

$$N_{\text{batch}} \times N_c \times N_c \times N_{\text{bins}}.$$

- Only the selected upper-triangular entries are filled; the redundant conjugate entries are left unused.
- The result is normalized by reference auto-spectra:

$$P_{c_1 c_2}(b) \leftarrow \frac{P_{c_1 c_2}(b)}{\sqrt{P_{c_1 c_1}^{\text{ref}}(b)} \sqrt{P_{c_2 c_2}^{\text{ref}}(b)}}.$$

Planck backend: non-periodic maps

The note `pbk_power_spectrum_note.pdf` describes the two backend branches.

- If `PBC=True`, the estimate stays in Fourier space.
- If `PBC=False`, the backend first inverse-transforms the binned map,

$$X_{c_1}^{(b)}(x) = \mathcal{F}^{-1} \left[M_b(k) \hat{X}_{c_1}(k) \right] (x),$$

then averages

$$X_{c_1}^{(b)}(x) \overline{X_{c_2}(x)}$$

with a bin-dependent real-space taper/crop mask.

- So the intended object is still a radial auto/cross power spectrum; the non-periodic branch changes the estimator to reduce boundary artifacts.

Scattering2d.py: what is called power

In `Scattering2d.py`, the main single-field power term is

$$P_{00}(j, \ell) = \langle |I * \psi_{j,\ell}|^2 \rangle_x.$$

- This is wavelet-band energy, not a separately binned radial FFT spectrum.
- It is included directly in `for_synthesis` as

$$\log P_{00}(j, \ell).$$

- The isotropic vector uses

$$P_{00}^{\text{iso}}(j) = \frac{1}{L} \sum_{\ell} P_{00}(j, \ell).$$

- A newer extra term `P00g` also computes energy after filtering by Gaussian concentric rings, and appends $\log P_{00g}$ to the non-isotropic synthesis vector.

Scattering2d.py: cross-field terms

The two-field routine computes auto powers for each field:

$$P_{00,a}(j, \ell), \quad P_{00,b}(j, \ell).$$

It also computes the normalized cross-band correlation

$$\text{Corr}_{00}(j, \ell) = \frac{\left\langle \widehat{a}(k) \overline{\widehat{b}(k)} |\psi_{j,\ell}(k)|^2 \right\rangle_k}{\sqrt{P_{00,a}(j, \ell) P_{00,b}(j, \ell)}}.$$

- Therefore auto and cross information can both appear.
- But this interface is pairwise: it takes two fields a, b , not an arbitrary $N_c \times N_c$ channel-pair mask.
- The flattened vector contains $P_{00,a}$, $P_{00,b}$, and real and imaginary parts of Corr_{00} .

Main difference

Current backend

- Separate PS block.
- Radial Fourier bins $M_b(k)$.
- Multi-channel auto/cross tensor.
- Pair selection controlled by one channel-pair mask.
- Non-periodic mode uses a real-space taper/crop estimator.

Scattering2d.py

- Power terms are embedded in scattering covariance vectors.
- Main term is wavelet-band energy P_{00} .
- Optional P00g uses Gaussian rings.
- Cross-field power appears as normalized pairwise correlation Corr_{00} .
- No general multi-channel PS tensor.

Practical takeaway

- The backend `st_compute_ps` constraint matches radial auto/cross Fourier spectra directly.
- The `Scattering2d.py` implementation constrains band energies and band correlations attached to the wavelet scattering representation.
- Both include auto-power information.
- Cross-power differs most:
 - backend: selected entries of a full channel-pair cross-spectrum tensor;
 - `Scattering2d.py`: pairwise normalized cross-correlations for two input fields.
- The backend is closer to an explicit power-spectrum loss; the older code is closer to adding power-like scattering covariance coordinates.